Meeting 5B — Instrument Layer & Audio Layer Architecture

Digital Harmonium Workstation

Phase 3.5 — Musical Processing Architecture

Purpose

This meeting defines the software architecture responsible for transforming a keyboard press into audible sound.

The previously defined Hardware Layer and Runtime Layer manage the physical device and overall system operation. This document focuses exclusively on the modules that perform musical interpretation and digital signal processing.

The architecture is divided into two major layers:

- Instrument Layer (Musical Understanding)
- Audio Layer (Digital Sound Processing)

Together these seventeen modules form the complete synthesis engine of the workstation.

Layer 3 — Instrument Layer

The Instrument Layer understands musical concepts.

It knows about:

- instruments
- presets
- regions
- samples
- key mapping
- velocity mapping
- musical parameters

It does **not** generate audio.

Module 1 — Instrument Manager

Purpose

The Instrument Manager owns the currently loaded VHL instrument and provides global access to its musical contents.

Only one instrument is active at a time.

Responsibilities

- Load selected VHL instrument
- Maintain instrument metadata
- Expose preset list
- Manage instrument-wide transpose
- Provide access to region database
- Notify other modules when instrument changes

Inputs

- Instrument selection from UI
- Loaded VHL file

Outputs

- Active instrument
- Instrument information
- Global configuration

Module 2 — Preset Manager

Purpose

Manages presets contained inside one VHL instrument.

Future instruments may contain multiple playable presets.

Example:

- Harmonium
- Organ
- Accordion
- Strings

The Preset Manager determines which preset is currently active.

Responsibilities

- Select preset
- Change preset
- Maintain active preset
- Notify Region Mapper when preset changes

Inputs

- User preset selection

Outputs

- Active preset

Module 3 — Region Mapper

Purpose

One of the most important modules.

Converts a keyboard event into the correct playable region.

Example:

Input:

Key = C4

Velocity = 94

Output:

Region #8

Sample #2

Envelope

Filter

Loop settings

Pitch parameters

Responsibilities

- Key range lookup
- Velocity lookup
- Region selection
- Multiple region support
- Round-robin support (future)
- Layer support (future)

Inputs

- MIDI key
- Velocity
- Active preset

Outputs

- Selected region
- Associated parameters

Module 4 — Sample Library

Purpose

Owns all PCM samples currently loaded into RAM.

It never plays sound.

It only stores sample data.

Responsibilities

- Sample pointers
- PCM buffers
- Sample rate
- Channel count
- Loop points
- Sample length

Inputs

- VHL Loader

Outputs

- Sample references

Module 5 — Parameter Database

Purpose

Stores every musical parameter imported from the VHL instrument.

Every newly created voice copies its initial parameters from this module.

Stored Parameters

Volume

Pan

Root Key

Fine Tune

Transpose

Filter Cutoff

Filter Resonance

Attack

Decay

Sustain

Release

Loop Mode

Velocity Range

Key Range

LFO Settings

Reverb Send

Chorus Send

Future Effects

Responsibilities

Provide immutable reference parameters to every new voice.

Instrument Layer Summary

The Instrument Layer converts:

Keyboard Event

↓

Playable Region

↓

Sample Reference

↓

Musical Parameters

Nothing here produces sound.

It only prepares data for the Audio Layer.

Layer 4 — Audio Layer

The Audio Layer performs all real-time digital signal processing.

This layer executes continuously while audio is playing.

It receives playable regions from the Instrument Layer and transforms them into digital audio.

Module 6 — Voice Allocator

Purpose

Creates voices whenever notes are played.

Every active note corresponds to one active voice.

Responsibilities

- Allocate voices
- Release voices
- Voice stealing
- Polyphony management

Inputs

- Key press
- Region information

Outputs

- Active voice

Module 7 — Voice Processor

Purpose

Updates every active voice every audio block.

A voice remains alive until its release stage finishes.

Responsibilities

- Maintain playback position
- Update playback state
- Advance voice lifetime
- Detect completed voices

Inputs

- Active voices

Outputs

- Updated voices

Module 8 — Sample Reader

Purpose

Reads PCM data from sample memory.

Responsibilities

- Read PCM
- Loop handling
- End detection
- Playback position

Inputs

- Voice playback position

Outputs

- Raw PCM samples

Module 9 — Pitch Engine

Purpose

Determines playback speed.

Pitch is calculated using musical information.

Uses

Root key

Pressed key

Transpose

Fine tuning

Pitch bend

Future modulation

Outputs

Playback increment

Module 10 — Interpolator

Purpose

Converts fractional playback positions into smooth audio.

Without interpolation pitch shifting introduces audible artifacts.

Responsibilities

- Linear interpolation
- Higher-order interpolation (future)

Outputs

Interpolated sample values

Module 11 — Envelope Engine

Purpose

Calculates amplitude evolution over time.

Every voice owns an independent envelope.

Responsibilities

Attack

Decay

Sustain

Release

Envelope completion

Outputs

Current amplitude multiplier

Module 12 — LFO Engine

Purpose

Produces low-frequency modulation.

Supported Targets

Pitch Vibrato

Amplitude Tremolo

Filter Modulation

Future Parameters

Outputs

Current modulation values

Module 13 — Filter Engine

Purpose

Processes samples through digital filters.

Responsibilities

Low-pass filtering

Cutoff frequency

Resonance

Future filter types

Outputs

Filtered samples

Module 14 — Effects Engine

Purpose

Applies global audio effects after voice processing.

Initial Effects

Reverb

Chorus

Future Effects

Delay

Equalizer

Compressor

Stereo Enhancement

Module 15 — Mixer

Purpose

Combines every active voice into one audio stream.

This module is the heart of the synthesizer.

Responsibilities

Mix voices

Apply voice volume

Apply pan

Prevent overflow

Prepare stereo output

Inputs

All active voices

Outputs

Mixed stereo stream

Module 16 — Limiter / Output Stage

Purpose

Final protection stage before transmission to hardware.

Responsibilities

Prevent clipping

Normalize output

Maintain safe amplitude

Outputs

Final PCM stream

Module 17 — DMA Buffer Builder

Purpose

Builds the exact audio blocks consumed by the ESP32 I²S DMA engine.

Responsibilities

Fill DMA buffer

Maintain continuous playback

Synchronize audio timing

Hand completed blocks to I²S

Outputs

DMA-ready audio buffer

Complete Audio Processing Pipeline

The complete musical processing chain becomes:

Keyboard Scanner

↓

Region Mapper

↓

Voice Allocator

↓

Voice Processor

↓

Sample Reader

↓

Pitch Engine

↓

Interpolator

↓

Envelope Engine

↓

LFO Engine

↓

Filter Engine

↓

Effects Engine

↓

Mixer

↓

Limiter

↓

DMA Buffer Builder

↓

MAX98357 DAC / Amplifier

↓

Speaker / Headphones

Final Architecture Summary

The complete synthesis engine consists of **17 dedicated software modules**:

Instrument Layer (5)

1. Instrument Manager
2. Preset Manager
3. Region Mapper
4. Sample Library
5. Parameter Database

Audio Layer (12)

1. Voice Allocator
2. Voice Processor
3. Sample Reader
4. Pitch Engine
5. Interpolator
6. Envelope Engine
7. LFO Engine
8. Filter Engine
9. Effects Engine
10. Mixer
11. Limiter / Output Stage
12. DMA Buffer Builder

Each module has a single, well-defined responsibility, making the architecture modular, maintainable, extensible, and suitable for real-time execution on the ESP32 while remaining faithful to the VHL instrument specification.